PaisaPilot: A Modular Personal Finance Management System with Explainable AI

Sofia Francis, Hiya Navalakha, Saihajvir Singh, Jatin Motwani and Shivani Mittal
Department of Computer Engineering, Mukesh Patel School of Technology Management and Engineering, SVKM's NMIMS
Mumbai, India

***Abstract*—Random train/test splits can overstate the accuracy of transaction categorisation models in personal finance applications, because transactions of the same merchant appear in both partitions. This paper presents PaisaPilot, a modular AI-based personal finance system for Indian users that combines categorisation, forecasting, anomaly and subscription detection, a health score, optical character recognition (OCR) and retrieval-augmented generation (RAG) assistance, with rationales and a feedback-as-labels loop. On the same 59,891-transaction subset of a 67,121-transaction synthetic dataset, accuracy falls from 99.94% under a random split to 75.53% under merchant-grouped evaluation (weighted F1 from 99.94% to 75.65%); a pilot on one real bank statement gives only 40.2% on 373 held-out rows. The Isolation Forest reaches an AUC-ROC of 0.918, the best of twenty forecasters a weekly MAPE of 37.9% (87.3% accuracy on 30-day totals for per-user models and 25.9% four-week error under a stricter rolling-origin test), and the subscription classifier 100% recall and 95.4% precision against current database labels. All 164 backend tests pass. PaisaPilot is a research prototype, not a production platform.**

***Index Terms*—Personal finance management, transaction categorisation, leakage-aware evaluation, anomaly detection, spending forecasting, explainable AI, retrieval-augmented generation**

I. Introduction

Personal finance applications in India increasingly categorise UPI, card and bank transactions with machine learning. Users benefit only if categories are accurate on unseen merchants, yet most systems report accuracy on a random split [1], [2], [3]. With many near-identical rows per merchant, such a split lets the classifier memorise merchant strings instead of learning category semantics. Such leakage is well documented in machine learning [3], [4] but has not been quantified for merchant-based transaction categorisation.

Motivation. Reported accuracy rarely states how the split was built, so systems cannot be compared; existing prototypes give the user a bare label or number without a reason [5], [6], [7]; and most evaluations use synthetic data without measuring the gap between synthetic and real data.

Objectives. (1) Build PaisaPilot, a modular system integrating categorisation, forecasting, anomaly and subscription detection, health scoring, OCR and RAG assistance; (2) quantify how much a random split overstates categorisation accuracy relative to a merchant-grouped split on identical data; (3) attach a rationale to outputs and turn user corrections into training labels; (4) run a first pilot on real, anonymised bank data and report how synthetic-trained models behave there.

Contributions. (a) A merchant-grouped categorisation evaluation shown beside the random-split result (99.94% vs. 75.53% accuracy). (b) A design where each categorisation, anomaly and recommendation output shows its reason, and user corrections are saved as training labels. (c) A working system with 164 backend tests and a sandbox Account Aggregator flow, with its limits stated. (d) A real-data test setup with a fixed split by payee, a first real-data result (40.2%) and some trial improvements that we did not add to the live system.

II. Literature Review

Personal finance and categorisation. WONGA [7] read transactions from SMS, processed bill images and set categories with Naive Bayes, but did not explain each prediction. Sahel et al. [6] made a rule-based web tracker with no explanation layer. Talasila [8] tested MyFinanceAI with 1,000 users but did not measure explainability. Naik et al. [5] described tracking and budgeting ideas without explaining each decision. Kharat et al. [9] used BERT for categories, reinforcement learning for budgets and LSTM for forecasts on synthetic data, but did not split the data by merchant.

Forecasting, anomalies and recommendation. Kulshrestha et al. [1] used regression and LSTM forecasting on a small group of users without strong baselines; Manasa et al. [2] checked their linear-regression forecasts only with functional tests. Kharat et al. [9] applied Isolation Forest [10] to savings ratios. Uddin et al. [11] found a linear SVM (91.59%) narrowly ahead of a transformer (89.49%) on Bangla-English fraud messages. Forecasts are scored with MAPE, sMAPE and MAE [12] against baselines such as Prophet [13], Holt-Winters [14] and robust regression [15].

Explainability, RAG and leakage. El Amine El Alami et al. [16] reviewed 22 computational-finance studies from 2024 and 2025: SHAP [17] and feature importance are an emerging trend but not a standard property, and LIME [18] is the other common method. Retrieval-augmented generation [19] grounds answers in retrieved passages. Leakage between training and test data inflates reported performance [3], [4], and grouped cross-validation is the standard remedy [20].

Research gap. Table I summarises the systems: standard metrics are reported, but end-user interpretability is untested [1], [2], [5], [6] and none uses a merchant-grouped protocol. PaisaPilot addresses this gap.

IEEE Conference ID: 73082

TABLE I
REVIEWED SYSTEMS COMPARED WITH PAISAPILOT

Work	Main techniques	Explanations	Grouped split
[7]	SMS extraction, Naive Bayes	No	No
[6]	Rule-based tracker	No	No
[8]	Gradient boosting, deep networks	Not measured	No
[5]	Tracking and budgeting	No	No
[9]	BERT, RL, LSTM, Isolation Forest	No	No
[1]	Regression, LSTM	Not reported	No
[2]	Linear regression	Not reported	No
Ours	TF-IDF + SVM, IF, RF, 20 forecasters, RAG	Rationales	Yes

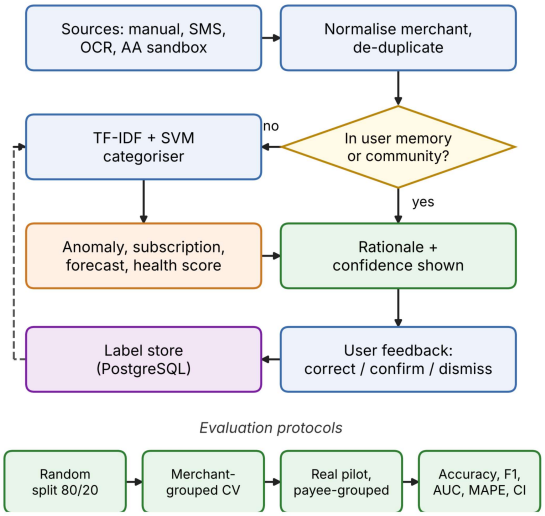


Fig. 1. Workflow of the proposed system: transaction pipeline with correction layers (top), explanation and feedback loop (middle) and evaluation protocols (bottom). The dashed arrow is the supervised-retraining loop.

III. PROPOSED METHODOLOGY

A. Overview and Architecture

Fig. 1 shows the complete workflow. Transactions enter by manual entry, pasted UPI or bank SMS, receipt OCR or a sandboxed Account Aggregator (AA) flow, are normalised, and are categorised after two correction layers. The categorised history feeds the anomaly, subscription, forecasting and health-score modules; each output carries a rationale, and user feedback is stored as labels for supervised retraining.

Fig. 2 shows the four-layer architecture. The client layer holds 26 screens in English, Hindi and Marathi, React context providers for authentication, data, theme, language and currency, and a single API module through which every network call passes with its bearer token. The API layer is a FastAPI application that registers 28 routers exposing about 149 route handlers. The analytics layer holds the categoriser, forecasting service, anomaly and subscription detectors, health score and knowledge base. The persistence layer holds the PostgreSQL database, trained model files and offline training scripts, with

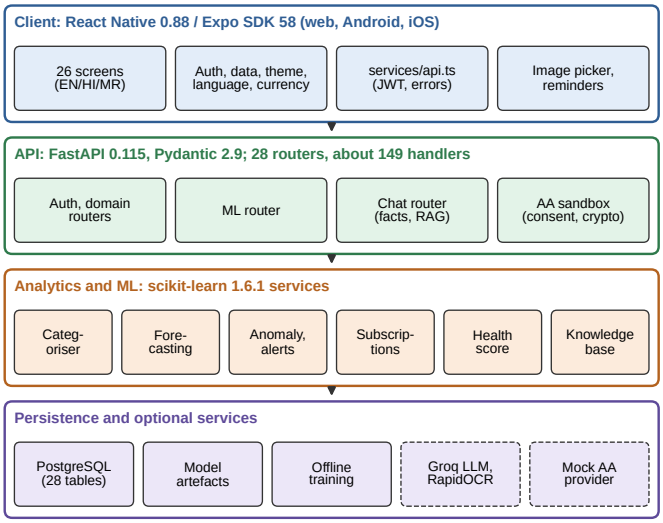


Fig. 2. Four-layer architecture of PaisaPilot: client, API, analytics and machine learning, and persistence with optional external services (dashed).

optional external services (a Groq LLM, RapidOCR and a mock AA provider).

The database has 28 tables (25 core and 3 for the AA sandbox), nearly all owned by a user. The transactions table stores, besides merchant, amount, payment method and date, the AI fields (category, confidence, reason, confirmation flag, model version) and provenance fields (source and external reference), with a partial unique index that blocks duplicate AA imports. A training-labels table records every user decision that teaches a model.

B. Datasets

The synthetic dataset has 67,121 transactions of 14 demonstration users with 6,137 unique merchants, spanning April 2022 to September 2026. About 900 curated merchant entries were added, 404 placeholder merchants renamed, 127 contradictory labels corrected and 42,579 rows re-dated uniformly. For merchant variety, the synthetic Augmented IndiaTransact-MultiFacet2024 dataset (about 10,000 rows) was mapped to the 15 categories. Both datasets are synthetic and contain no real banking records.

The real-statement pilot dataset is one user's bank statement for April to October 2026: 928 raw rows, 778 after cleaning, 458 distinct narration keys and 407 payee groups. It cannot be redistributed. All rows were labelled with the 15 categories by an LLM-assisted procedure that used no PaisaPilot model (300 high-, 135 medium- and 343 low-confidence labels).

C. Transaction Categorisation

Two correction layers are consulted before the model: a private per-user merchant memory and a global community registry that pools corrections across users by majority vote (Algorithm 1). Otherwise the model classifies the text "clean merchant, payment method, amount bin, lexicon tokens", where lexicon tokens come from a hand-built gazetteer of 520

Algorithm 1 Categorisation of one transaction $t = (m, a, p)$ of user u

$m' \leftarrow \textsc{Normalise}(m)$
if $m' \in \textsc{UserMemory}[u]$ **then**
    return (stored category, high confidence, "your correction")
else if $m' \in \textsc{Community}$ **then**
    return (majority category, high confidence, "community")
end if
$x \leftarrow \textsc{TfIdf}(m' \| p \| \textsc{AmountBin}(a) \| \textsc{Lexicon}(m'))$
$c \leftarrow \arg\max_k \mathbf{w}_k^\top x$; $s \leftarrow$ decision margin; $r \leftarrow$ matched evidence
return (c, s, r)

Indian merchant words. Features are sublinear TF-IDF weights over word 1-2-grams (8,000 features) and character 2-4-grams (12,000 features), fed to a one-vs-rest linear SVM [21], [22] that minimises, per class,

$$\tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \max\left(0, 1 - y_i \mathbf{w}^\top \mathbf{x}_i\right)^2, \qquad C = 2, \qquad (1)$$

with balanced class weights.

D. Anomaly and Subscription Detection

An unsupervised Isolation Forest [10] and a supervised Random Forest [23] use six features: log amount, per-user amount z-score, deviation from the user's per-merchant mean, log merchant frequency, share of total spending and weekday. Tree count (150 to 400) and contamination (0.01 to 0.1) were selected by AUC-ROC against 58 known anomaly labels. The supervised layer is a 400-tree Random Forest whose positives are user-reported anomalies and seeded statistical outliers and whose negatives are dismissed alerts and sampled unflagged transactions. Real positive examples are few, so each one is copied nine times with small Gaussian noise (the weekday is not changed), and cross-validation scores use the real rows only. Flags are shown to the user as unusual activity to review, never as fraud.

To find recurring payments, we measure how much the amounts, the gaps between payments and the day of the month vary (coefficient of variation). Points are given for frequency (up to 25), stable amounts (up to 20), regular gaps (up to 20), the same day of the month (10), time span (up to 15), a subscription or bills category (12) and a known brand (12), with a maximum of 114. A group with 50 points or more is flagged, and a known brand with two or more payments always gets at least 52 points, so a plan change does not hide a real subscription. A Random Forest learns from the subscriptions users confirmed or dismissed, using eight features per merchant (number of payments, amount variation, share of similar amounts, median gap, gap regularity, fit to weekly, monthly, quarterly or yearly cycles, known-brand flag and mean log amount).

E. Forecasting, Health Score, RAG and OCR

Forecasting. Twenty methods are tested on weekly spending, and for each user the method with the lowest holdout MAPE is used and then checked on six 30-day blocks. The methods are simple baselines; ordinary, Huber [15] and IQR-clipped regression; ridge regression with extra features; Holt-Winters [14]; Prophet [13]; regressions on past weeks; robust level estimates; and three ensembles. The amount shown in the app blends the last 13 weeks with the last 52 weeks (weights 0.6 and 0.4), and the per-user model sets the pattern across weeks.

TABLE II
COMPONENTS OF THE FINANCIAL HEALTH SCORE

Component	Max	Rule
Budget use	25	Best near 65% use; lower above 80% and above 100%; 10 if no budget
Savings rate	25	25 at 30% or more; scaled down below that; lower if negative; 5 if no income
Anomaly density	20	20 if none; 15 if under 5% of transactions; 10 if under 10%; lower above that
Goal progress	15	10 times overall progress, plus up to 5 for active goals; 3 if no goals
Regularity	15	15 times (1 minus half the capped CV of daily counts); 8 if very few transactions

Health score. The score is a fixed formula, not a trained model. It adds five component scores, rounds the total and keeps it between 0 and 100 (Table II). Ratings are Excellent ($\geq$80), Good (60), Fair (40), Needs Attention (20) and Critical ($<$20). The home screen uses a simpler rule: it starts at 50 and adds points for savings rate, budget use, few active alerts and a daily logging streak, up to 100.

RAG assistant. Questions about the user's own records are answered from database rows by a fixed-rule facts engine. Other questions use a prompt that adds a curated knowledge base (61 passages, eight collections) embedded with all-MiniLM-L6-v2 [24], with the top four passages retrieved from a FAISS flat L2 index [25], and pass it to an LLM through the Groq API with automatic model fallback.

OCR. Receipts are read with RapidOCR; regular expressions extract merchant, total, date and tax, a category is chosen by keyword scoring, the date is checked and duplicate receipts are removed.

Explainability and feedback. Outputs show the merchant match and confidence, the deviation that triggered an anomaly, the health-score components, a forecast's typical error and range, or the sources of a chat reply. User corrections, confirmations and dismissals are stored as labels for supervised retraining. SHAP [17] and LIME [18] are not implemented.

IV. Implementation Details

The client is a React Native 0.88.1, Expo SDK 58 and TypeScript 6.0 code base that runs on the web, Android and iOS; the backend uses Python 3.9 to 3.13 with FastAPI 0.115.0, SQLAlchemy 2.0.35 and Pydantic 2.9.0 over PostgreSQL; machine learning uses scikit-learn 1.6.1 [20]. Seeds are fixed (42; 1337 for real-data splits). Table III lists the system scale and hyper-parameters. Bank synchronisation is a sandboxed AA simulation with a consent life-cycle and X25519, HKDF-SHA256 and AES-256-GCM payload encryption; a licensed AA is out of scope.

TABLE III
SYSTEM SCALE AND MODEL HYPER-PARAMETERS

Item	Setting
Scale	26 screens (EN, HI, MR); 28 routers, about 149 handlers; 28 tables; 164 backend tests; JWT and bcrypt
Categoriser	Word TF-IDF 1-2-grams (8k) and character 2-4-grams (12k), sublinear; LinearSVC $C = 2$, balanced; 15 classes; three-fold GroupKFold on merchant
Isolation Forest	200 trees, contamination 0.01, six features
Random Forests	400 trees, balanced; anomaly (six features) and subscription (eight features)
Forecaster	20 methods; last $H \leq 13$ weeks held out; per-user lowest MAPE; validated 13/52-week level for display
RAG / OCR	all-MiniLM-L6-v2, FAISS flat L2, $k = 4$, Groq LLM; RapidOCR

The SMS scanner reads UPI and bank messages that the user pastes and shows them for review, so the app does not need wide SMS permissions. Sandbox bank linking runs an eight-step consent flow: request consent, approve, create an ephemeral X25519 session, fetch encrypted data, decrypt, import with duplicate-safe external references, categorise and revoke.

Backend tests use pytest and FastAPI's test client against a throwaway PostgreSQL database; each test registers its own users and checks behaviour, including data isolation, through the public API. Static checks are the TypeScript compiler and ESLint, and eight frontend unit tests cover the payload builder and optimistic sync. Security measures are hashed passwords, signed expiring tokens, per-user scoping, consent checks, in-memory decryption keys, duplicate-safe imports and sanitised error messages. Data is not encrypted at rest, there is no multi-factor authentication or independent audit, and the system is deployed only locally. Offline, the client shows cached data and keeps new transactions visible with an unsynced marker; automatic re-sending is not implemented.

V. EXPERIMENTAL SETUP, RESULTS AND DISCUSSION

A. Protocols and Metrics

Categorisation. A 59,891-transaction subset is evaluated by (i) a random stratified split (47,912 train, 11,979 test, 15 categories) and (ii) three-fold merchant-grouped cross-validation, in which all transactions of a merchant fall in one fold; configuration selection also uses grouped folds.

Real pilot. A manifest was frozen before tuning: five folds grouped on a cleaned payee key, two held out, giving 405 training rows (243 payees) and 373 test rows (164 payees) with no overlap, guarded by a SHA-256 data hash. Macro-F1 covers the seven classes with enough test support; accuracy has 95% Wilson intervals [26].

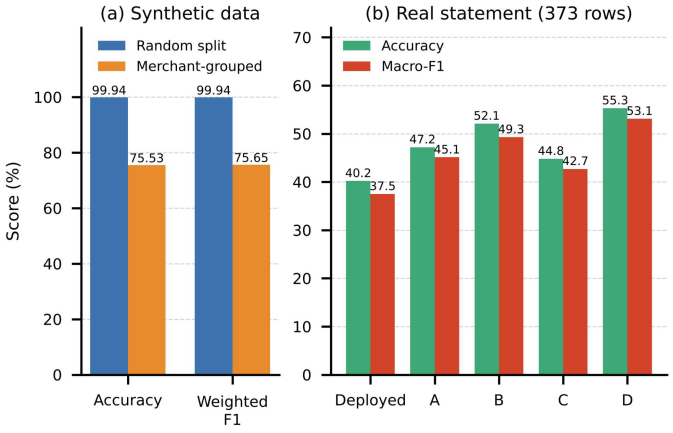


Fig. 3. (a) Categorisation under random-split and merchant-grouped evaluation on identical synthetic data; (b) accuracy and macro-F1 on the frozen real-statement test split for the deployed model and exploratory variants A to D.

Other tasks. Forecasts are scored per user on a chronological split (last $H \leq 13$ weeks) by

$$\text{MAPE} = \frac{100}{n}\sum_t \frac{|y_t - \hat{y}_t|}{|y_t|}, \quad \text{sMAPE} = \frac{100}{n}\sum_t \frac{|y_t - \hat{y}_t|}{(|y_t| + |\hat{y}_t|)/2}, \tag{2}$$

plus MAE in rupees [12], and by six-fold 30-day-block and calendar-month backtests.

B. Categorisation

The random split gives 99.94% accuracy and weighted F1, but merchant-grouped cross-validation gives 75.53% accuracy and 75.65% weighted F1 (standard deviation 0.025), a drop of about 24 points (Fig. 3a). The likely cause is merchant-level leakage, which lets the model learn merchant text rather than category associations. Of four configurations compared under grouped folds, the selected one (word 1-2-grams plus character 2-4-grams, $C = 2$) scored 75.65% weighted F1 against 74.79%, 74.66% and 74.58% for word-only variants; these differences are smaller than the fold-to-fold standard deviation (about 2.5 points).

Real-statement pilot. The deployed model, trained only on synthetic data, classifies 150 of 373 held-out real rows correctly (40.2%, 95% CI 35.4 to 45.3; macro-F1 0.375), well below the synthetic grouped hold-out (72.2% vs. 81.6% for the deployed recipe retrained on the same split). Fig. 3b compares the deployed model with four exploratory variants not promoted to production.

C. Anomaly Detection

The Isolation Forest was trained on 53,391 expense transactions and reaches an AUC-ROC of 0.918 against the 58 known labels, flagging 534 transactions (1.0%). AUC varies only between 0.913 and 0.918 across tree counts, within run-to-run variation, and contamination does not change AUC because it only sets the flagging threshold (534 flags at 0.01, 5,339 at 0.10). The supervised classifier is trained on 770 rows (35 real positives, 315 augmented positives, 420 negatives) and

TABLE IV
SELECTED FORECASTING METHODS ON A CHRONOLOGICAL HOLDOUT (MEAN OVER 14 USERS)

Method	MAPE	sMAPE	WMAPE	MAE (Rs.)
Huber (raw)	37.9	28.3	28.5	66,023
Clip linear (IQR)	40.0	28.9	29.1	67,408
Mean naive	40.9	44.3	40.4	95,005
Prophet	42.7	32.0	31.6	72,774
Last naive	52.3	38.0	38.7	87,916

TABLE V
ROLLING-ORIGIN RE-EVALUATION OF THE DEPLOYED FORECASTER

Measure	Result
Lowest-error method (10 tested)	Recent/52-week blend
Four-week WAPE / MAE	25.9% / Rs. 163,609
Median user / worst user	25.3% / 32.8%
Four-week mean / last week only	32.5% / 47.7%
Users where blend beats mean	14 of 14
80% range coverage	78% of 2,510

scored on real rows only: stratified five-fold cross-validation gives AUC 0.993 and average precision 0.894, and merchant-grouped five-fold cross-validation over 351 groups gives 0.995, 0.937 and F1 0.848 (Fig. 4a). Because augmented near-copies of every real positive enter each training fold, these scores are likely optimistic.

D. Forecasting

Twenty methods were trained on the earlier weeks of each of the 14 users and evaluated on the last H (4 to 13) weeks. Huber regression on the raw weekly series gives the lowest error (Table IV): MAPE 37.9%, sMAPE 28.3% and MAE Rs. 66,023. Prophet and Holt-Winters, which model seasons directly, did not do better than simple robust regression on these short and noisy series. The mean-naive baseline has a competitive MAPE but far higher sMAPE and MAE, so more than one metric is needed.

The product shows a 30-day projection rather than weekly values. Choosing the best method per user lowers mean weekly MAPE to 31.2% (sMAPE 27.4%, MAE Rs. 46,103), slightly optimistically because selection uses the same holdout. Summed over four weeks, errors partly cancel: the per-user models were 87.3% accurate on four-week totals and 83.9% on calendar months.

Rolling-origin re-evaluation. Every week after an eight-week warm-up was a forecast origin for each user (up to 225 origins per user). The blend of the recent and 52-week level had the lowest error of ten methods (Table V), 25.9% WAPE on four-week totals against 32.5% for the plain four-week mean, and beat that mean for all 14 users. An 80% range from the 10th and 90th percentiles of each user's earlier actual-to-predicted ratios contained 78% of 2,510 real four-week totals, with a half-width of about 46%. This protocol is not directly comparable with the 87.3% block figure.

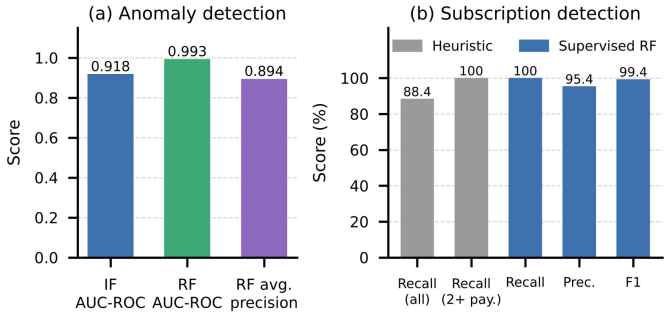


Fig. 4. (a) Anomaly detection: Isolation Forest AUC-ROC and Random Forest AUC-ROC and average precision (stratified CV); (b) subscription detection, heuristic vs. supervised Random Forest.

E. Subscription Detection and Feedback

The heuristic scorer was applied to all 25,186 user-merchant groups and flags 1,544 (6.1%), 747 with high confidence. Of 704 known-brand groups, 82 contain a single payment, which no recurrence detector should flag; this caps recall at 88.4% (622 of 704), and recall is 100% on groups with two or more payments, partly by construction because of the known-brand floor. The supervised classifier is trained on 3,701 rows (249 real positives, 464 baseline negatives, 2,988 augmented rows; 88 real positive merchants) and has merchant-grouped five-fold AUC 1.000, average precision 1.000 and F1 0.994. Against the database it recovers all 249 gold merchants among 261 flagged (recall 100%, precision 95.4%; Fig. 4b); the 12 remaining are unreviewed candidates rather than verified errors. The classifier is tested on merchants it was trained on, so these numbers show agreement with the current labels more than performance on new merchants. For the 79 correction records (17 merchants, 14 users), lookups after the memory and registry steps were 100% consistent; this shows that the mechanism works but does not measure accuracy.

F. Health Score, Assistant and Testing

The health score is a fixed formula, so it has no training or test error. Over the 14 users the mean is 54.4 (SD 3.4, range 47 to 62), and every user is rated Fair or Good; anomaly density and budget use add the most points, and savings and goals the least. The small spread suggests that the synthetic users are alike, and the weights were not checked against an outside benchmark; the score is useful because it is easy to understand.

We did not measure retrieval quality or answer correctness with numbers, so the assistant was checked only for function: the knowledge base and index load correctly, questions about the user's records are answered by the facts engine, other questions get a prompt with sources, and four tests confirm that error details never reach the user. Rationales are clear statements that can be checked, such as "From your previous activity with MERCHANT".

The backend suite (29 modules) passed all 164 cases and the TypeScript check found no errors; 67 cases cover the AA sandbox. Passing tests show that the endpoints work as

designed, not that the predictions are accurate, so model results are reported separately.

G. Threats to Validity and Comparison

Synthetic data may be cleaner than real spending (the real-statement test setup helps here); copies of positive examples in training make the anomaly-classifier CV look better than it is; the subscription classifier is tested on data it was trained on, against a gold set that includes brands confirmed automatically; and no user study checked whether users understand the reasons shown. A numerical comparison with the reviewed works is not valid: Uddin et al. [11] classify fraud messages, Kharat et al. [9] report LSTM accuracy (63%) on synthetic data rather than MAPE, Talasila [8] gives no figures, and no merchant-grouped benchmark was found. Qualitatively (Table I), PaisaPilot (i) reports categorisation under a leakage-aware protocol, quantifying a 24-point random-split optimism, (ii) compares 20 forecasters with naive, statistical and robust baselines, (iii) attaches rationales and persists feedback as labels and (iv) provides a first real-data baseline (40.2%). Its drawbacks are narrow single-user real evidence and rule-based explanations compared with [16].

VI. Conclusion and Future Scope

PaisaPilot is a modular AI-driven personal finance system for Indian users. Random-split accuracy (99.94%) overstates the merchant-grouped result (75.53%), and on a first real statement the synthetic-trained model reaches only 40.2%. The Isolation Forest reaches an AUC of 0.918, the best weekly forecaster a MAPE of 37.9% (87.3% on 30-day totals for per-user models and 25.9% four-week error under the stricter rolling test), and the subscription classifier 100% recall and 95.4% precision against current database labels.

Limitations. Most results use synthetic data for 14 users, and the only real evidence is one user's statement with LLM-assisted labels. Bank synchronisation is a sandbox, OCR is not benchmarked, recommendations are seeded or rule-based, and there is no security audit or A/B test. Weekly forecasting remains hard, the anomaly classifier has only 35 real positives and explanations are rule-based.

Applications and ethics. PaisaPilot suits students and early-career UPI users as a personal tool and is a worked example of how the evaluation protocol changes reported performance; it gives no licensed financial advice. Privacy was a design constraint (synthetic data, pasted SMS, per-user scoping, revocable consent), and anomalies are shown as unusual activity, not fraud.

Future scope. We will evaluate on multi-user real data with human-verified labels, finish the pre-registered ablation on a fresh real split, add SHAP- or LIME-style attributions with a user study, and benchmark OCR and connect to a licensed Account Aggregator.

Data and code availability. Code: https://github.com/Saihaj0/PaisaPilot. The real statement is not distributed.

Use of AI tools. An AI assistant helped with formatting, language editing and figures; the authors reviewed all content.

References

[1] N. Kulshrestha *et al.*, "Real-time expense tracking and financial forecasting through AI," *Int. J. Sci. Res. Eng. Manag.*, vol. 9, no. 5, 2025.

[2] H. Manasa *et al.*, "AI based personal finance management system," *Int. J. Eng. Res. Technol.*, vol. 14, no. 8, 2025.

[3] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, pp. 15:1-15:21, 2012.

[4] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[5] L. Naik *et al.*, "Automating financial management," in *Proc. IEEE BITCON-24*, 2024.

[6] S. Sahel *et al.*, "Streamlining personal finances: an ExpenseTracker website study," in *Proc. IEEE SPARC*, 2024.

[7] M. Uyanahewa *et al.*, "WONGA: the future of personal finance management," in *Proc. IEEE INCET*, 2023.

[8] S. Talasila, "AI-driven personal finance management," *Int. Res. J. Eng. Technol.*, vol. 11, no. 7, 2024.

[9] A. Kharat *et al.*, "A smart AI-based personal finance assistant," *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 13, no. 11, pp. 167-180, 2025.

[10] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE ICDM*, 2008, pp. 413-422.

[11] M. Uddin *et al.*, "Multilingual financial fraud detection using machine learning," arXiv:2603.11358, 2026.

[12] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *Int. J. Forecast.*, vol. 22, no. 4, pp. 679-688, 2006.

[13] S. J. Taylor and B. Letham, "Forecasting at scale," *Am. Stat.*, vol. 72, no. 1, pp. 37-45, 2018.

[14] P. R. Winters, "Forecasting sales by exponentially weighted moving averages," *Manag. Sci.*, vol. 6, no. 3, pp. 324-342, 1960.

[15] P. J. Huber, "Robust estimation of a location parameter," *Ann. Math. Stat.*, vol. 35, no. 1, pp. 73-101, 1964.

[16] S. El Amine El Alami, S. Mouiha, A. Hafid, and A. El Hilali Alaoui, "Machine learning and deep learning in computational finance: a systematic review," arXiv:2511.21588, 2025.

[17] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. NIPS*, 2017, pp. 4765-4774.

[18] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you? Explaining the predictions of any classifier," in *Proc. ACM SIGKDD*, 2016, pp. 1135-1144.

[19] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020, pp. 9459-9474.

[20] F. Pedregosa *et al.*, "Scikit-learn: machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011.

[21] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273-297, 1995.

[22] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, "LIBLINEAR: a library for large linear classification," *J. Mach. Learn. Res.*, vol. 9, pp. 1871-1874, 2008.

[23] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5-32, 2001.

[24] N. Reimers and I. Gurevych, "Sentence-BERT: sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982-3992.

[25] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535-547, 2021.

[26] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Am. Stat. Assoc.*, vol. 22, no. 158, pp. 209-212, 1927.